

Fiche de révision — Théorie des programmes (S7)

Méthodes clés pour traiter n'importe quel exercice de sémantique/calculabilité/complexité

1. Sémantique des programmes — les trois approches

Sémantique	Idée	Utilité typique
Opérationnelle	décrit l'exécution pas à pas via des règles de transition $\blacksquare \text{instr, état} \blacksquare \rightarrow \text{état}'$	prouver qu'un programme calcule bien ce qu'on attend, étape par étape
Dénotationnelle	associe à chaque programme une fonction mathématique (état $\rightarrow$ état)	raisonner sur le programme sans parler d'exécution
Axiomatique (Hoare)	décrit l'effet d'une instruction via des assertions logiques avant/après	prouver la correction d'un programme

2. Logique de Hoare — méthode pas à pas

Un triplet de Hoare $\{P\} S \{Q\}$ signifie : si P est vrai avant d'exécuter S , alors Q est vrai après (en supposant que S termine $\rightarrow$ correction partielle).

Règles à appliquer selon la construction

Construction	Règle à appliquer
Affectation $x := e$	$\{Q[e/x]\} x := e \{Q\}$ — remplacer x par e dans Q pour trouver la précondition la plus faible
Séquence $S1; S2$	$\{P\} S1 \{R\}$ et $\{R\} S2 \{Q\} \blacksquare \{P\} S1; S2 \{Q\}$ — chercher l'assertion intermédiaire R
Conditionnelle if C then $S1$ else $S2$	$\{P \wedge C\} S1 \{Q\}$ et $\{P \wedge \neg C\} S2 \{Q\} \blacksquare \{P\} \text{if } C \text{ then } S1 \text{ else } S2 \{Q\}$
Boucle while C do S	$\{I \wedge C\} S \{I\} \blacksquare \{I\} \text{while } C \text{ do } S \{I \wedge \neg C\}$ — I est l'invariant de boucle : vrai avant, pendant et après chaque itération
Affaiblissement/renforcement	si $P \blacksquare P'$ et $\{P'\} S \{Q'\}$ et $Q' \blacksquare Q$, alors $\{P\} S \{Q\}$

Trouver un invariant de boucle — méthode

- 1) Regarder ce que la boucle est censée accomplir au final (la postcondition Q).
- 2) Formuler une propriété toujours vraie qui, combinée à la négation de la condition d'arrêt, donne Q .
- 3) Vérifier l'invariant en 3 points : il est vrai avant la boucle (initialisation) ; il est préservé par un tour de boucle (préservation) ; combiné à $\neg C$ il implique Q (utilité).

Correction partielle vs correction totale

Correction partielle : « si ça termine, le résultat est correct ». Correction totale = correction partielle + preuve de terminaison (trouver une expression entière positive qui décroît strictement à chaque itération, un « variant de boucle »).

3. Calcul de plus faible précondition (wp)

- $wp(x := e, Q) = Q[e/x]$
- $wp(S1; S2, Q) = wp(S1, wp(S2, Q))$
- $wp(\text{if } C \text{ then } S1 \text{ else } S2, Q) = (C \blacksquare wp(S1, Q)) \wedge (\neg C \blacksquare wp(S2, Q))$

- Méthode d'exercice : partir de la postcondition demandée et remonter le programme instruction par instruction (de la fin vers le début) en appliquant ces règles.

4. Calculabilité

- **Machine de Turing** : modèle abstrait de calcul (ruban infini, tête de lecture/écriture, table de transitions) servant de référence pour définir ce qui est « calculable ».
- **Thèse de Church-Turing** : toute fonction « calculable de façon intuitive/algorithmique » est calculable par une machine de Turing.
- **Problème indécidable** : aucun algorithme ne peut le résoudre pour toutes les entrées. Exemple central : le **problème de l'arrêt** (savoir si un programme donné termine sur une entrée donnée) est indécidable — à citer comme résultat de référence.
- **Méthode de preuve d'indécidabilité (réduction)** : pour montrer qu'un problème B est indécidable, montrer qu'un problème déjà connu indécidable (souvent l'arrêt) peut se ramener (se réduire) à B ; si B était décidable, l'arrêt le serait aussi → contradiction.

5. Complexité algorithmique

Classe	Définition	Exemple
P	problèmes de décision résolubles en temps polynomial	tri, plus court chemin
NP	problèmes dont une solution proposée se vérifie en temps polynomial	SAT, coloration de graphe, voyageur de commerce (version décision)
NP-complet	dans NP, et tout problème de NP s'y réduit en temps polynomial	SAT (Cook-Levin), 3-SAT, clique, sac à dos (décision)

Méthode pour prouver qu'un problème est NP-complet

- 1) Montrer qu'il est dans NP : donner un certificat et montrer qu'il se vérifie en temps polynomial.
- 2) Choisir un problème déjà connu NP-complet (souvent 3-SAT ou un problème proche du sujet).
- 3) Construire une réduction polynomiale de ce problème connu vers le problème étudié (transformer une instance de l'un en instance de l'autre en préservant la réponse oui/non).
- 4) Conclure : NP + réduction depuis un NP-complet ■ NP-complet.

P = NP ?

Question ouverte : on ne sait pas si tout problème vérifiable rapidement est aussi résoluble rapidement. Un problème NP-complet résolu en temps polynomial impliquerait $P = NP$.

6. Pièges classiques en examen

- Dans une preuve de Hoare sur une boucle, oublier l'un des 3 points de vérification de l'invariant (initialisation / préservation / utilité).
- Confondre correction partielle et correction totale : la terminaison doit être prouvée séparément.
- Pour une réduction NP-complétude, réduire dans le mauvais sens (il faut réduire un problème connu vers le nouveau, pas l'inverse).
- Confondre indécidable (aucun algorithme ne le résout jamais) et NP-difficile (résoluble mais pas en temps polynomial connu) — ce sont deux notions différentes.